

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

D DHEERAJ REDDY (VTU25783)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Job Portal Management System" by D.DHEERAJ REDDY (VTU25783) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Hemamalini.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Professor & Head

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(D.DHEERAJ REDDY)

Date:06/05/2026

ABSTRACT

The Job Portal Management System is a web-based application that connects job seekers and employers on a single platform. It allows users to create profiles, upload resumes, search for jobs, and apply online. Employers can post job openings, view applications, and shortlist candidates.

The system is developed using Spring Boot, Spring Data JPA, and MySQL for backend operations, while the frontend uses HTML, CSS, and Thymeleaf. It provides a simple, efficient, and secure way to manage the recruitment process.

The system includes role-based access control for candidates and recruiters, ensuring secure and personalized user experiences. It supports real-time application tracking, allowing users to monitor the status of their job applications. Email notifications are integrated to inform users about application updates and important actions. The platform also features efficient search and filtering options to help candidates find relevant job opportunities quickly.

Keywords: Job Portal, Recruitment System, Spring Boot, Web Application, Job Seeker, Employer, Database Management

LIST OF FIGURES

1.1	System Architecture Diagram	3
3.1	Framework Architecture	7
4.1	Project Architecture	9
4.2	DATA FLOW DIAGRAM	10
5.1	Portal Home page	15
5.2	Candidate Registration Page	16
5.3		17

LIST OF TABLES

5.1	Comparison of Existing System vs Proposed System .	15
7.1	Mapping of Project to Complex Engineering Problem (CEP)	19
7.2	Mapping of Project to Complex Engineering Activities (CEA)	20
7.3	SDG Mapping for the Project	20

LIST OF ACRONYMS AND ABBREVIATIONS

CI	Continuous Integration
ORM	Object Relational Mapping
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
JPA	Java Persistence API
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
MVC	Model View Controller

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5

3.2	Backend Implementation	6
3.3	Database Implementation	8
4	METHODOLOGIES	9
4.1	Project Architecture	9
4.2	DataFlow Diagram	10
4.3	Module 1: User Management	11
4.4	Module 2: Job Management	11
4.5	Module 3: Application Management	12
5	RESULTS AND DISCUSSIONS	14
6	CONCLUSION AND FUTURE ENHANCEMENTS	18
6.1	Conclusion	18
6.2	Future Enhancements	18
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	19
7.1	Mapping of the Project to Complex Engineering Prob- lem (CEP)	19
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	20
7.3	SDG Mapping (CEP Section)	20

References	20
Source Code	23

Chapter 1

INTRODUCTION

1.1 Introduction

The Job Portal Management System is a web-based application designed to simplify the process of job searching and recruitment. In today's digital world, traditional methods of hiring and job searching are time-consuming and inefficient. This system provides a centralized platform where job seekers and employers can interact efficiently.

Job seekers can create profiles, upload resumes, search for jobs based on various criteria, and apply online. Employers can post job vacancies, manage job listings, and review applications. The system ensures faster communication and reduces manual effort in recruitment.

1.2 Aim of the Project

The main aim of this project is to develop a user-friendly and efficient job portal that connects job seekers and employers. It focuses on

simplifying the hiring process, improving accessibility, and providing a secure platform for managing job applications.

1.3 Scope of the Project

The scope of this project includes providing basic functionalities such as user registration, login, job posting, job searching, and application tracking. It is suitable for small to medium-scale recruitment systems and can be extended with advanced features like notifications, resume parsing, and analytics.

1.4 Framework

The system is developed using Spring Boot and follows the Model-View-Controller (MVC) architecture. The backend handles business logic and database operations using Spring Data JPA, while the frontend is designed using HTML, CSS, and Thymeleaf. Security is implemented using Spring Security to ensure safe authentication and authorization.

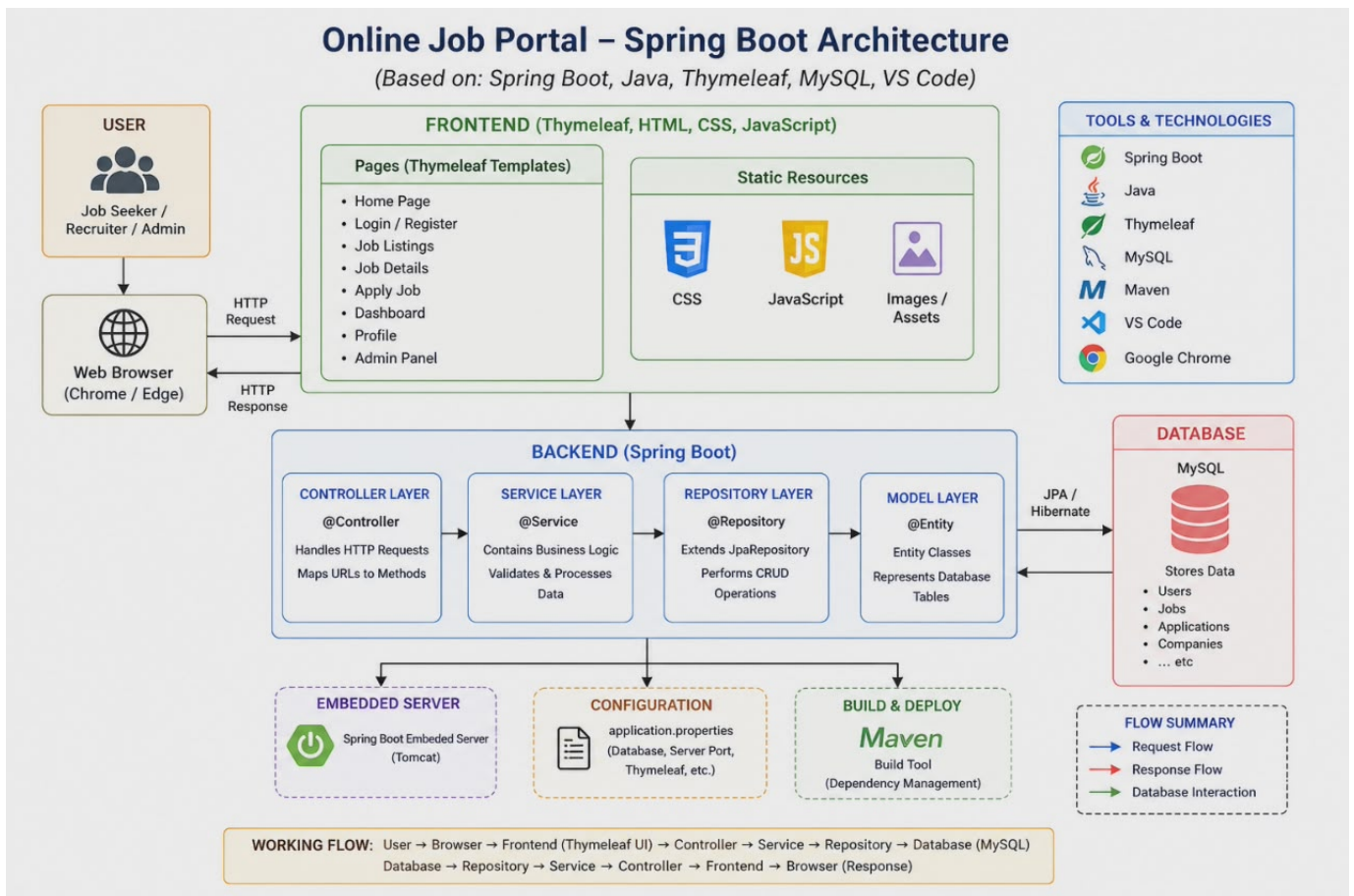


Figure 1.1: System Architecture Diagram

Figure 1.1 shows the system architecture of the Online Job Portal built using the Spring Boot MVC pattern. The client sends HTTP requests to the controller, which forwards them to the service layer for processing business logic. The service layer interacts with the repository layer through dependency injection to perform database operations. The model represents the data structure, and all data is stored and managed in the database using Spring Data.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

One of the most popular platforms is LinkedIn, which not only acts as a job portal but also as a professional networking site. It allows users to connect with recruiters, showcase their profiles, and receive job recommendations based on their skills and connections. It is especially useful for professional and high-level job opportunities.

Naukri is one of the leading job portals in India, known for its large database of resumes and job listings. It is widely used by employers for hiring across different industries and provides tools for resume building and job alerts. It mainly focuses on experienced professionals and large-scale recruitment.

Other platforms like Shine, Apna, and Foundit (formerly Monster) also provide job search and recruitment services with features such as quick job applications, local job listings, and user-friendly interfaces.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend is developed using HTML, CSS, and JavaScript with Thymeleaf templates. Tools like Visual Studio Code or Eclipse are used for development. A web browser is used to run and test the user interface.

3.1.2 Structure of the Project

- The project is divided into Frontend, Backend, and Database modules.
- The Frontend includes HTML, CSS, and JavaScript files for user interaction.
- The Backend is developed using Node.js/Express to handle application logic and APIs.

- The Database uses MySQL to store user, job, and application data.
- This structure ensures modularity, scalability, and easy maintenance.

3.1.3 Execution Procedure

The frontend of the application is executed as part of the Spring Boot framework. Once the backend server is started, the frontend is automatically loaded and rendered through the web browser.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the project is developed using Spring Boot with Java, and the entire application is implemented and executed using Visual Studio Code as the primary development environment.

3.2.2 Structure of the Project

The backend follows the Model-View-Controller (MVC) architecture. It is organized into controller, service, repository, and model packages. Controllers handle requests, services contain business logic, repositories manage database operations, and models represent data entities. This structure ensures better modularity and maintainability.

3.2.3 Execution Procedure

The backend is executed by running the main Spring Boot appli-

cation class. The server starts on localhost (default port 8080) and handles client requests. APIs can be tested using a web browser or Postman. The application processes requests and returns appropriate responses.

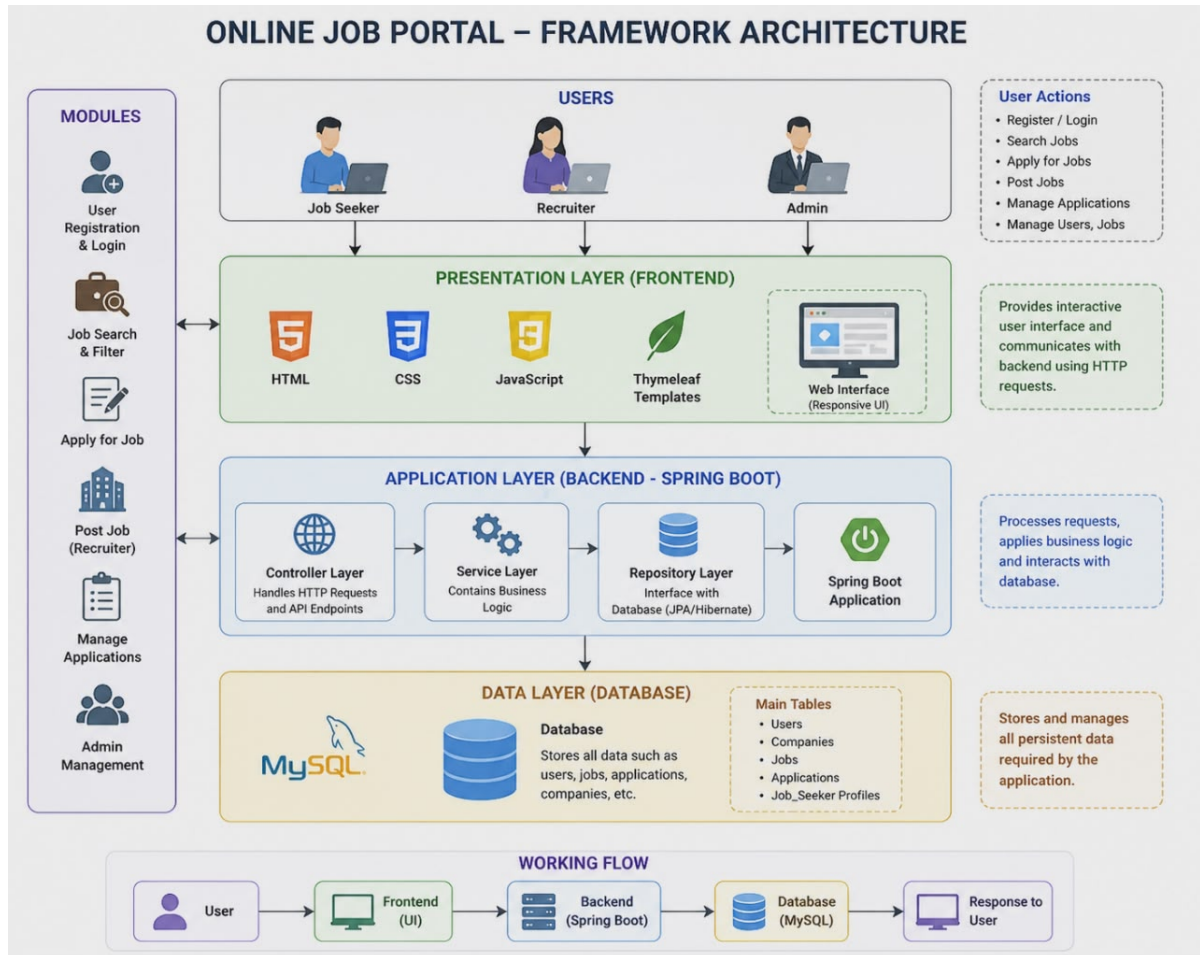


Figure 3.1: **Framework Architecture**

Figure 3.1 shows the layered framework architecture of the Online Job Portal System. The frontend interacts with the controller, which forwards requests to the service layer for processing. The repository layer handles database operations using Spring Data MongoDB, and all data is stored in MongoDB.

3.3 Database Implementation

3.3.1 Database Configuration

The database used is MySQL or H2. Configuration is done in the application.properties file, including database URL, username, password, and driver settings.

3.3.2 Schema Structure

The database schema consists of tables such as User, Job, and Application. Relationships are maintained using foreign keys to ensure data integrity.

3.3.3 Tables created

The main tables created are User (stores user details), Job (stores job information), and Application (stores job applications and status).

Chapter 4

METHODOLOGIES

4.1 Project Architecture

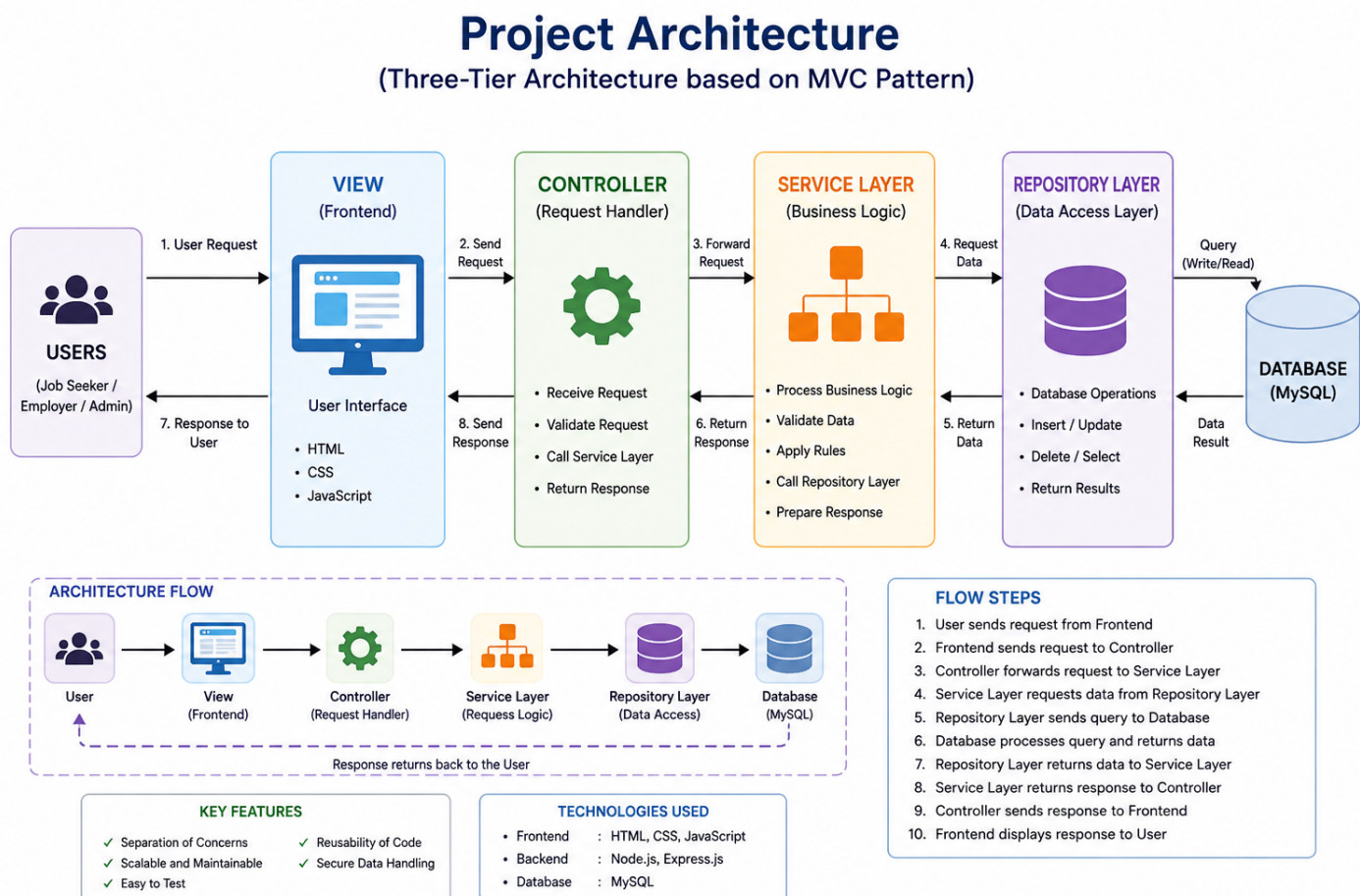


Figure 4.1: Project Architecture

Figure 4.1 Shows the project follows a three-tier architecture based on the MVC (Model-View-Controller) pattern. The frontend handles user interaction, the controller manages requests, the service layer processes business logic, and the repository layer interacts with the database.

4.2 DataFlow Diagram

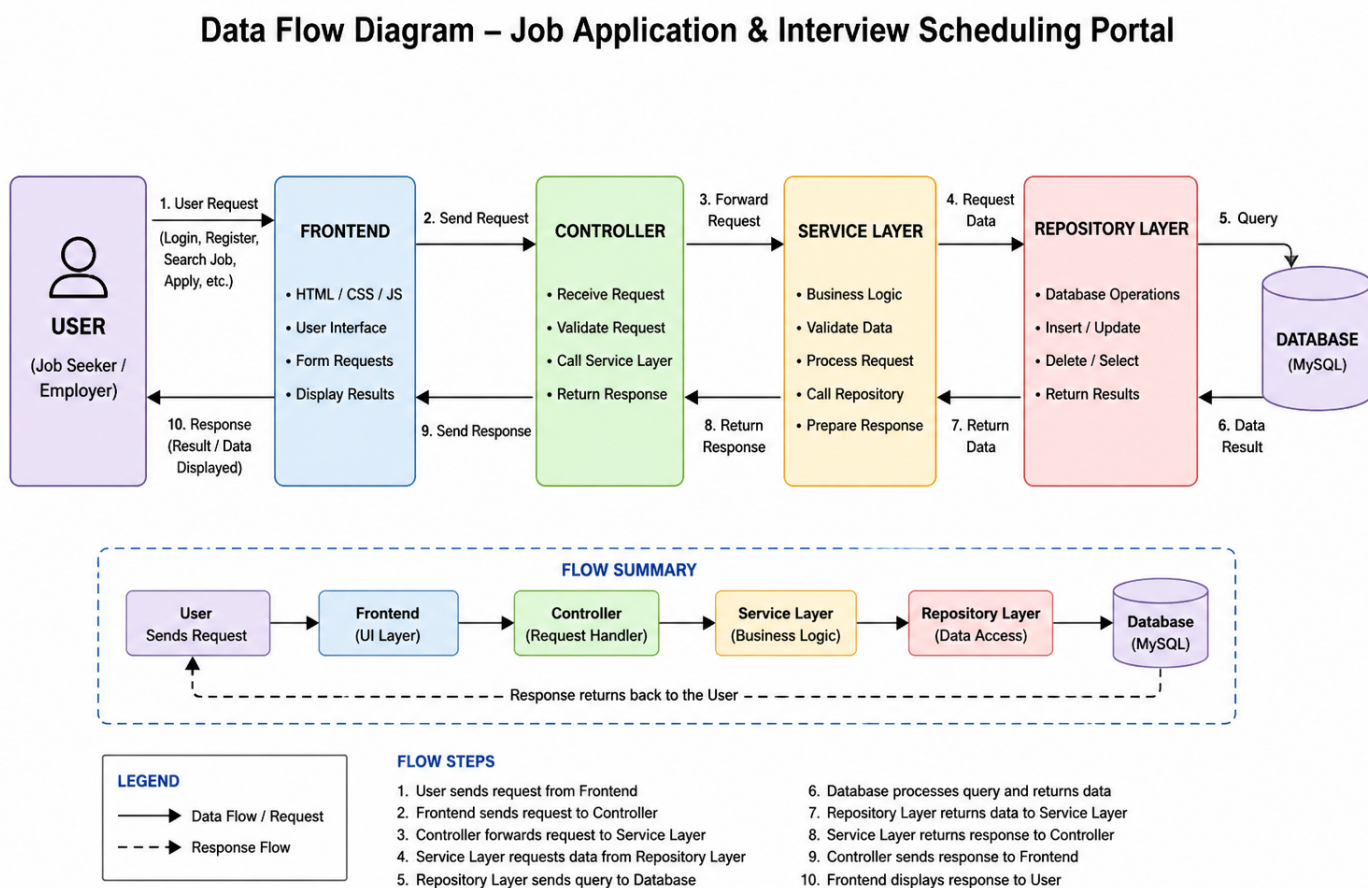


Figure 4.2: DATA FLOW DIAGRAM

Figure 4.2 Shows the data flow starts when a user sends a request through the frontend. The request is processed by the controller, passed to the service layer for logic execution, and then to the repository layer

for database operations. The response is sent back to the user through the same flow.

4.3 Module 1: User Management

The User Management module is responsible for handling user-related operations such as registration, login, and profile management. It allows both job seekers and employers to create accounts with secure authentication mechanisms. The module validates user credentials and ensures data integrity during login and signup processes. Users can update their personal information, upload resumes, and manage profile details. Role-based access control is implemented to differentiate between users, employers, and administrators. Passwords are securely stored and managed to prevent unauthorized access. The module also supports session handling to maintain user login states. It ensures smooth interaction between frontend forms and backend services. Overall, this module provides a secure and efficient way to manage user identities within the system.

4.4 Module 2: Job Management

The Job Management module enables employers to create, update, and delete job postings efficiently. It allows adding details such as

job title, description, location, salary, and required skills. Job seekers can browse and search available jobs using filters like category, location, and experience level. The module maintains all job-related data in the database for easy retrieval and updates. It also tracks job views and applications for analytical purposes. Employers can manage the status of job postings and monitor candidate applications. The system ensures only authorized users can modify job data. Integration with other modules allows seamless job application and interview scheduling. This module plays a key role in connecting job seekers with suitable employment opportunities.

4.5 Module 3: Application Management

This module manages job applications. Job seekers can apply for jobs, and employers can view applications, shortlist, or reject candidates.

Advantages of this Project

This project provides an efficient and user-friendly platform for recruitment and job searching.

a) Easy Access: Users can access job information anytime and anywhere.

b) Time Saving: Reduces manual effort in job searching and hiring.

c) Centralized System: All job-related activities are managed in one platform.

Disadvantages

The system depends on internet connectivity and may not include advanced features like AI-based recommendations in its basic version.

Content booktabs

Chapter 5

RESULTS AND DISCUSSIONS

The Job Portal Management System was successfully developed and tested. The application provides functionalities such as user registration, login, job posting, job searching, and application management. Both job seekers and employers were able to perform their tasks efficiently through the system.

The system interface is user-friendly and responsive. Job seekers can easily search and apply for jobs, while employers can post jobs and manage applications. The results show that the system reduces manual effort and improves the recruitment process.

Screenshots of the application interfaces such as login page, job listings, and application status are included below.

Table 5.1 compares the existing system with the proposed system. The proposed system provides advanced filtering, automated application tracking, a user-friendly interface, real-time notifications, and centralized data management, improving overall efficiency.

Table 5.1: Comparison of Existing System vs Proposed System

Feature	Existing System	Proposed System
Job Search	Limited	Advanced Filtering
Application Tracking	Manual	Automated
User Interface	Complex	User-Friendly
Notifications	Limited	Real-Time
Data Management	Decentralized	Centralized

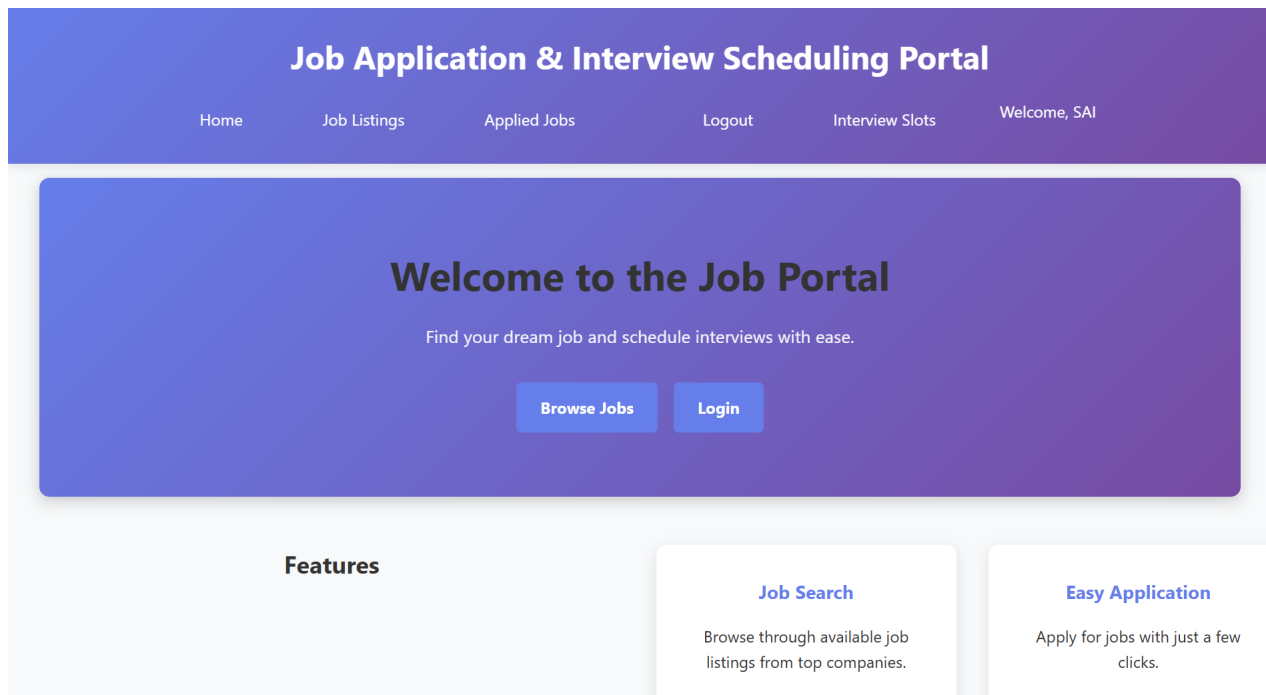


Figure 5.1: Portal Home page

Figure 5.1 shows the portal dashboard page of the Online Job Portal System. It provides a search bar for job titles, skills, or locations, along with options for candidate and recruiter access. The interface is designed to be simple and user-friendly for easy navigation. It also displays quick access sections for browsing jobs and managing user profiles. Users can easily switch between different functionalities through the navigation menu. The layout ensures a smooth user

experience with clear visibility of important features.

The screenshot displays the 'Candidate Registration Page' within a 'Job Portal' interface. The page features a purple header bar with the 'Job Portal' logo on the left and navigation links for 'Dashboard', 'Browse Jobs', 'Profile', and 'Logout' on the right. The main content area is a light gray box containing a white registration form titled 'Personal Information'. The form is organized into several sections: 'First Name' (DHEERAJ) and 'Last Name' (REDDY) are in the top row; 'Email' (vtu25783@veltech.edu.in) and 'Phone' (9550161065) are in the second row; 'Location' (CHENNAI,INDIA) and 'Years of Experience' (0) are in the third row. Below these is a 'Bio' section with a text area containing the placeholder 'Tell us about yourself...'. At the bottom of the form is a prominent purple button labeled 'Save Profile'.

Figure 5.2: Candidate Registration Page

Figure 5.2 shows the candidate registration page of the Online Job Portal System. It allows users to enter personal details such as name, email, mobile number, and password, along with OTP verification for secure account creation. The interface is designed to be simple and user-friendly for easy registration. It also includes validation checks to ensure accurate and complete user information. Users receive instant feedback during the registration process for any input errors. The secure verification process enhances trust and prevents unauthorized access.

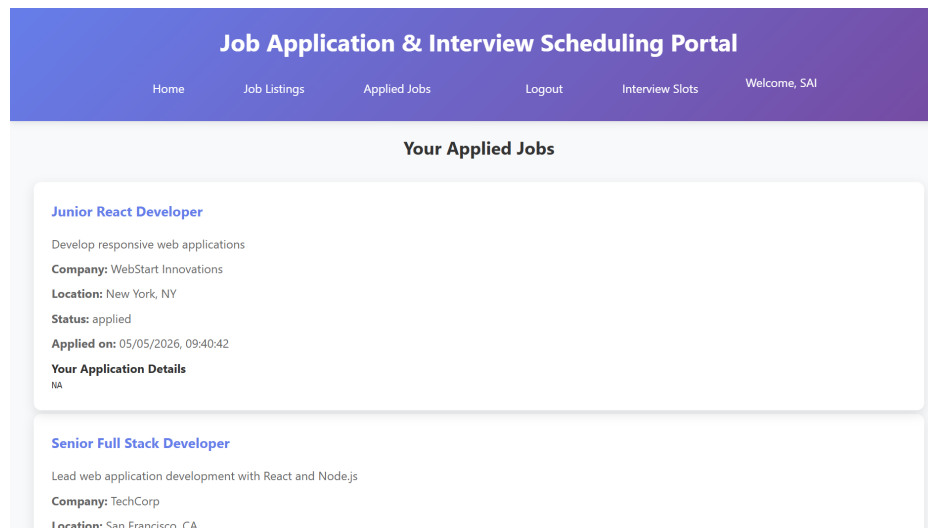


Figure 5.3:

Figure 5.3 “Applied Jobs” section of the portal, where users can view their submitted applications. It displays details such as job title, company, location, application date, and status. The layout is simple and organized, allowing users to easily track their application progress. Navigation options provide quick access to other features like job listings and interview slots.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Management System has been successfully designed and implemented to simplify the recruitment process. It provides an efficient platform for job seekers to search and apply for jobs, and for employers to post job openings and manage applications. The system reduces manual effort, improves accessibility, and ensures better communication between users.

6.2 Future Enhancements

The system can be further improved by adding advanced features such as AI-based job recommendations, resume parsing, and skill matching. Email and mobile notifications can be integrated for real-time updates. The application can also be extended into a mobile app and deployed on cloud platforms for better scalability and accessibility.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	In-depth engineering knowledge	WK3–WK6	Uses Spring Boot, MVC, Security, JPA, and DB design for multi-role job portal
WP2	Conflicting Requirements	Technical & non-technical conflicts	WK4–WK6	Balances secure role-based access with user-friendly workflows
WP3	Depth of Analysis	Analytical and design thinking	WK3–WK5	Involves ER modeling, API design, file upload, and email integration
WP4	Familiarity	Partially new problems	WK4, WK8	Job tracking and multi-role interaction beyond basic CRUD systems
WP5	Applicable Codes	Limited standard solutions	WK6	Custom logic for resume storage, tracking, and filtering
WP6	Stakeholder Needs	Multiple stakeholders	WK5–WK7	Supports job seekers, recruiters, and admins efficiently
WP7	Interdependence	System-level integration	WK3–WK6	Integrates frontend, backend, DB, security, and email services

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Diverse technologies	Uses Spring Boot, MVC, JPA, Thymeleaf, Security, and Java-MailSender
EA2	Interactions	Complex modules	Handles jobs, applications, authentication, resume upload, notifications
EA3	Innovation	Modern technologies	Role-based dashboards, filtering, and automated email updates
EA4	Societal Impact	User impact	Improves employment access and reduces manual hiring effort
EA5	Familiarity	Beyond standard	Combines authentication, tracking, and resume management

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance
SDG 8	Decent Work and Economic Growth	Connects job seekers with employers and improves career opportunities
SDG 9	Industry & Innovation	Promotes digital recruitment and automation
SDG 10	Reduced Inequalities	Provides equal job access through a centralized platform
SDG 16	Strong Institutions	Ensures secure and transparent system with role-based access

Table 7.3: SDG Mapping for the Project

References

- [1] Spring Boot Documentation. Available:
<https://spring.io/projects/spring-boot>

- [2] Thymeleaf Documentation. Available:
<https://www.thymeleaf.org/documentation.html>

- [3] Hibernate ORM Documentation. Available:
<https://hibernate.org/orm/documentation/>

- [4] MySQL Documentation. Available: <https://dev.mysql.com/doc/>

- [5] Apache Maven Project Documentation. Available:
<https://maven.apache.org/guides/>

- [6] Visual Studio Code Documentation. Available:
<https://code.visualstudio.com/docs>

- [7] Spring Initializr Tool. Available: <https://start.spring.io/>

- [8] REST API Tutorial – RESTful Web Services. Available:
<https://restfulapi.net/>

[9] Baeldung – Spring Boot Tutorials. Available:

<https://www.baeldung.com/>

[10] GeeksforGeeks – Java and Web Development Tutorials.

Available: <https://www.geeksforgeeks.org/>

Source Code

1. Home Page (index.html)

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Job Portal - Home</title>
7     <link rel="stylesheet" href="css/styles.css">
8 </head>
9 <body>
10     <header>
11         <h1>Job Application & Interview Scheduling Portal</h1>
12     </header>
13
14     <main>
15         <h2>Welcome to the Job Portal</h2>
16         <p>Find your dream job and schedule interviews with ease.</p>
17     </main>
18
19     <script src="js/common.js"></script>
20     <script src="js/home.js"></script>
21 </body>
22 </html>
```

2. Login Page (login.html)

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Login</title>
6 </head>
7 <body>
8   <h2>User Login</h2>
9   <form>
10     <input type="text" placeholder="Username" required>
11     <input type="password" placeholder="Password" required>
12     <button type="submit">Login</button>
13   </form>
14 </body>
15 </html>
```

3. Backend Server (app.js)

```
1 const express = require('express');
2 const bodyParser = require('body-parser');
3 const apiRoutes = require('./routes/api');
4 const { ready } = require('./db');
5
6 const app = express();
7 const PORT = process.env.PORT || 3000;
8
9 app.use(bodyParser.json());
10 app.use(bodyParser.urlencoded({ extended: true }));
11 app.use(express.static('client'));
12
13 ready.then(() => {
14   app.use('/api', apiRoutes);
15
16   app.listen(PORT, () => {
```

```
17     console.log('Server is running on http://localhost:${PORT}');
18   });
19 });
```

4. Database Schema (schema.sql)

```
1 CREATE TABLE Users (
2   id INT AUTOINCREMENT PRIMARY KEY,
3   username VARCHAR(50) NOT NULL UNIQUE,
4   password VARCHAR(255) NOT NULL,
5   email VARCHAR(100) NOT NULL UNIQUE,
6   role ENUM('user', 'admin', 'employer') DEFAULT 'user',
7   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8 );
9
10 CREATE TABLE Jobs (
11   id INT AUTOINCREMENT PRIMARY KEY,
12   title VARCHAR(100) NOT NULL,
13   description TEXT NOT NULL,
14   company VARCHAR(100) NOT NULL,
15   location VARCHAR(100) NOT NULL,
16   job_type ENUM('full-time', 'part-time', 'internship') DEFAULT 'full-time',
17   salary_min DECIMAL(10,2),
18   salary_max DECIMAL(10,2),
19   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
20 );
21
22 CREATE TABLE Applications (
23   id INT AUTOINCREMENT PRIMARY KEY,
24   user_id INT,
25   job_id INT,
26   status ENUM('applied', 'interview', 'selected', 'rejected') DEFAULT 'applied',
27   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
28   FOREIGN KEY (user_id) REFERENCES Users(id),
29   FOREIGN KEY (job_id) REFERENCES Jobs(id)
30 );
```

```
31
32 CREATE TABLE InterviewSlots (
33     id INT AUTO_INCREMENT PRIMARY KEY,
34     job_id INT,
35     slot_time DATETIME,
36     is_booked BOOLEAN DEFAULT FALSE,
37     FOREIGN KEY (job_id) REFERENCES Jobs(id)
38 );
39
40 CREATE TABLE Bookings (
41     id INT AUTO_INCREMENT PRIMARY KEY,
42     user_id INT,
43     slot_id INT,
44     FOREIGN KEY (user_id) REFERENCES Users(id),
45     FOREIGN KEY (slot_id) REFERENCES InterviewSlots(id)
46 );
```